

1) Fine tuning in api.testleaf.com

Create a Fine tuning dataset

1) Extract the Zip Folder -leafaptaps.zip

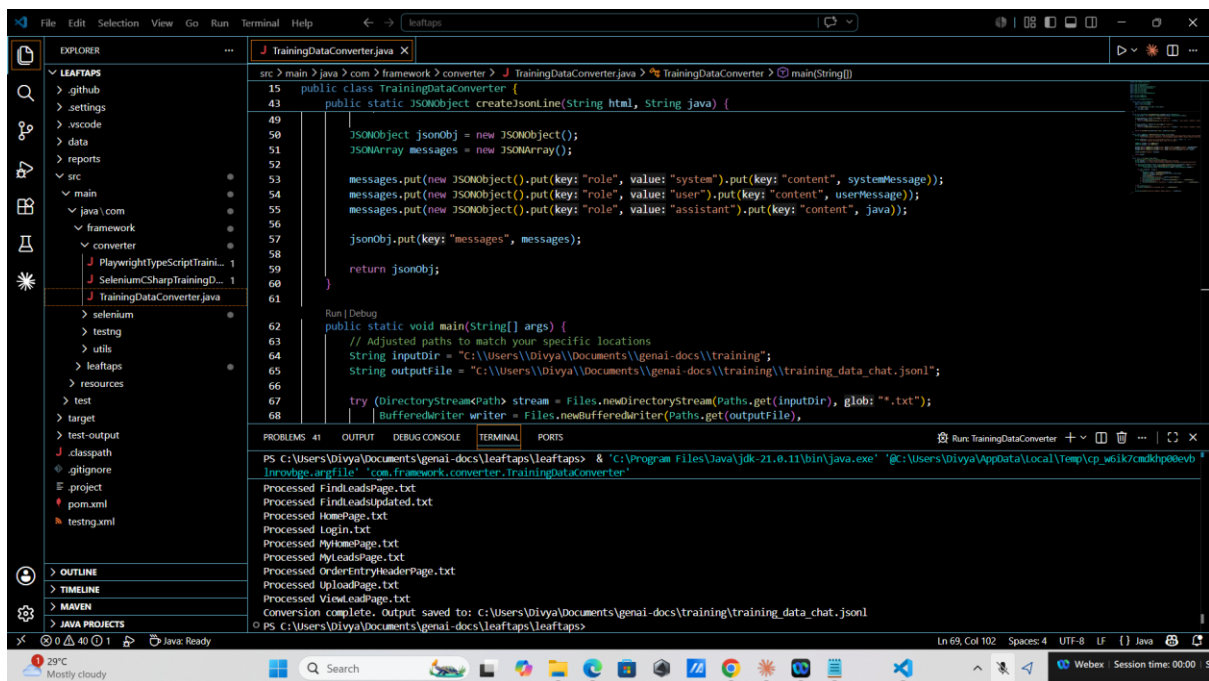
2) Import the leafaptaps folder in VS code

3) Change the input and output path

(src/main/framework/converter/TrainingDataConverter.java)

4) Run the java program

5) Check for the data in the training folder in jsonl format



The screenshot shows the Visual Studio Code interface with the file `TrainingDataConverter.java` open. The file is located at `src > main > java > com > framework > converter > TrainingDataConverter.java`. The code defines a `TrainingDataConverter` class with a `main` method that processes HTML files into JSONL format. The terminal output shows the execution of the program, which processes various HTML files (FindLeadsPage.txt, FindLeadsUpdated.txt, HomePage.txt, Login.txt, MyHomePage.txt, MyLeadsPage.txt, OrderEntryHeaderPage.txt, UploadPage.txt, ViewLeadPage.txt) and saves the output to `C:\Users\Divya\Documents\genai-docs\training\training_data_chat.jsonl`.

```
src > main > java > com > framework > converter > TrainingDataConverter.java > TrainingDataConverter > main(String[])
15 public class TrainingDataConverter {
43 public static JSONObject createJsonLine(String html, String java) {
49     JSONObject jsonObj = new JSONObject();
50     JSONArray messages = new JSONArray();
51
52     messages.put(new JSONObject().put(key: "role", value: "system").put(key: "content", systemMessage));
53     messages.put(new JSONObject().put(key: "role", value: "user").put(key: "content", userMessage));
54     messages.put(new JSONObject().put(key: "role", value: "assistant").put(key: "content", java));
55
56     jsonObj.put(key: "messages", messages);
57
58     return jsonObj;
59 }
60
61 Run | Debug
62 public static void main(String[] args) {
63     // Adjusted paths to match your specific locations
64     String inputDir = "C:\\Users\\Divya\\Documents\\genai-docs\\training";
65     String outputFile = "C:\\Users\\Divya\\Documents\\genai-docs\\training\\training_data_chat.jsonl";
66
67     try (DirectoryStream<Path> stream = Files.newDirectoryStream(Paths.get(inputDir), glob: "*.txt");
68         BufferedWriter writer = Files.newBufferedWriter(Paths.get(outputFile),
69             StandardCharsets.UTF_8)) {
70         for (Path htmlFile : stream) {
71             String javaCode = Files.readString(htmlFile, StandardCharsets.UTF_8);
72             JSONObject jsonObj = createJsonLine(Files.readString(htmlFile, StandardCharsets.UTF_8), javaCode);
73             writer.write(jsonObj.toJSONString() + "\n");
74         }
75     } catch (IOException e) {
76         e.printStackTrace();
77     }
78 }
```

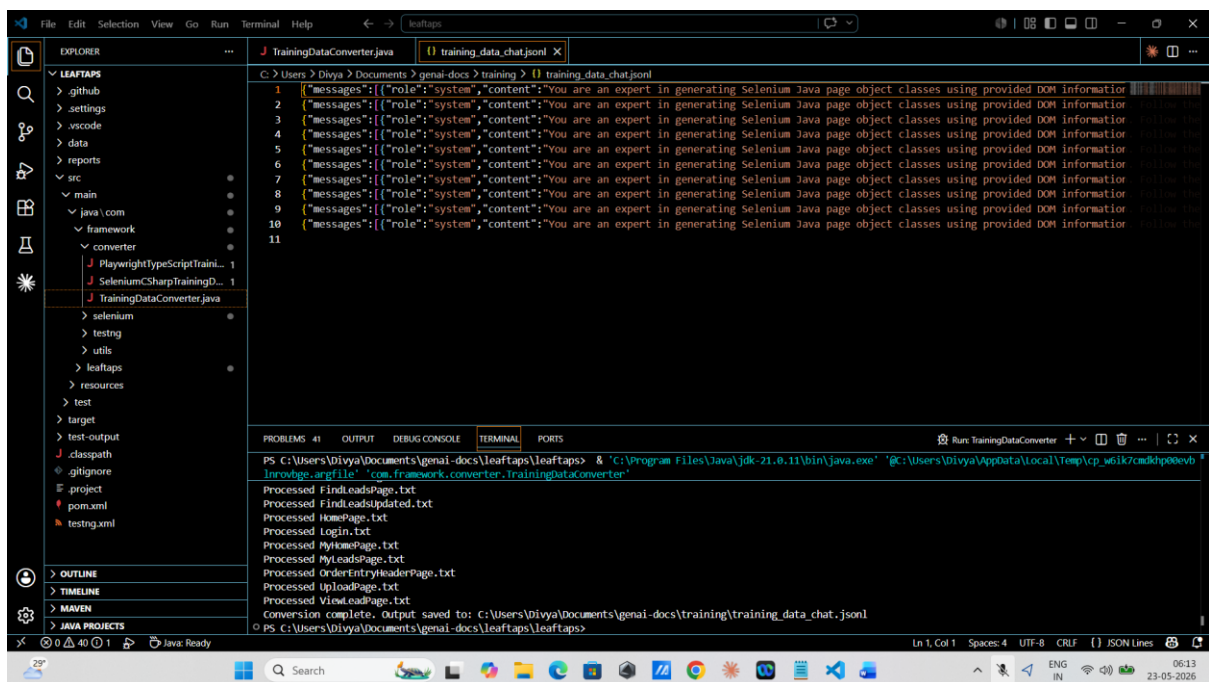
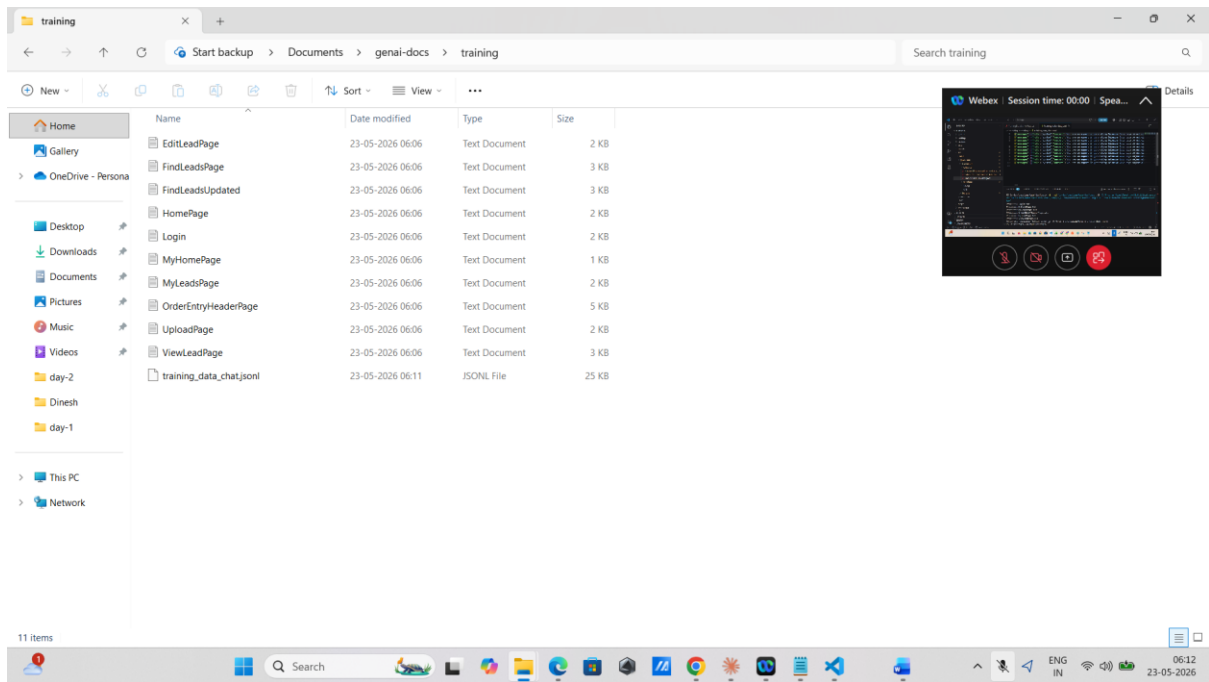
PROBLEMS 41 OUTPUT DEBUG CONSOLE TERMINAL PORTS

Run: TrainingDataConverter + - - - - -

PS C:\Users\Divya\Documents\genai-docs\leafaptaps> & 'C:\Program Files\Java\jdk-21.0.11\bin\java.exe' '@C:\Users\Divya\AppData\Local\Temp\cp_w6ik7cmdkhp00evb\lnrodgqe_argfile' 'com.framework.converter.TrainingDataConverter'

Processed FindLeadsPage.txt
Processed FindLeadsUpdated.txt
Processed HomePage.txt
Processed Login.txt
Processed MyHomePage.txt
Processed MyLeadsPage.txt
Processed OrderEntryHeaderPage.txt
Processed UploadPage.txt
Processed ViewLeadPage.txt
Conversion complete, output saved to: C:\Users\Divya\Documents\genai-docs\training\training_data_chat.jsonl

PS C:\Users\Divya\Documents\genai-docs\leafaptaps>



2) Fine tuning in api.testleaf.com

1) Prepare jsonl file with the converter (Create a new training data for createlead)

===HTML===

```
<input type="text" class="inputBox" name="companyName" size="30"
maxlength="100" id="createLeadForm_companyName">
```

```
<input type="text" class="inputBox" name="firstName" size="30" maxlength="100"
id="createLeadForm_firstName">

<input type="text" class="inputBox" name="lastName" size="30" maxlength="100"
id="createLeadForm_lastName">

<input type="submit" class="smallSubmit" name="submitButton" value="Create Lead"
id="ext-gen652">
```

```
===JAVA===
```

```
package com.leaf taps.pages;
```

```
import com.framework.selenium.api.design.Locators;
```

```
import com.framework.testng.api.base.ProjectSpecificMethods;
```

```
/**
```

```
 * Page object representing the Create Lead page.
```

```
 * Provides methods to create lead details and submit changes.
```

```
 * @author Auto-generated
```

```
 */
```

```
public class CreateLeadPage extends ProjectSpecificMethods {
```

```
    /**
```

```
     * Creates the company name in the Create Lead form.
```

```
     * @param updcomnyName The new company name to enter
```

```
     * @return The current CreateLeadPage instance
```

```
     */
```

```
    public CreateLeadPage EnterCompanyName(String entcomnyName) {
```

```
        Type(locateElement(Locators.ID,"createLeadForm_companyName"),
entcomnyName);
```

```

        reportStep(entcomnyName +"Company name is entered successfully ",
"pass");

        return this;

    }

```

```

    public CreateLeadPage EnterFirstName(String entfstName) {

        Type(locateElement(Locators.ID,"createLeadForm_firstName"),
entfstName);

        reportStep(entfstName +"First name is entered successfully ", "pass");

        return this;

    }

```

```

    public CreateLeadPage EnterLastName(String entlastName) {

        Type(locateElement(Locators.ID,"createLeadForm_lastName"),
entlastName);

        reportStep(entlastName +"First name is entered successfully ", "pass");

        return this;

    }

```

```

/**

```

```

 * Clicks the Create Lead button to submit the details.

```

```

 * @return A new ViewLeadPage instance

```

```

 */

```

```

    public ViewLeadPage clickSubmit() {

        click(locateElement(Locators.CLASS_NAME,"smallSubmit"));

        return new ViewLeadPage();

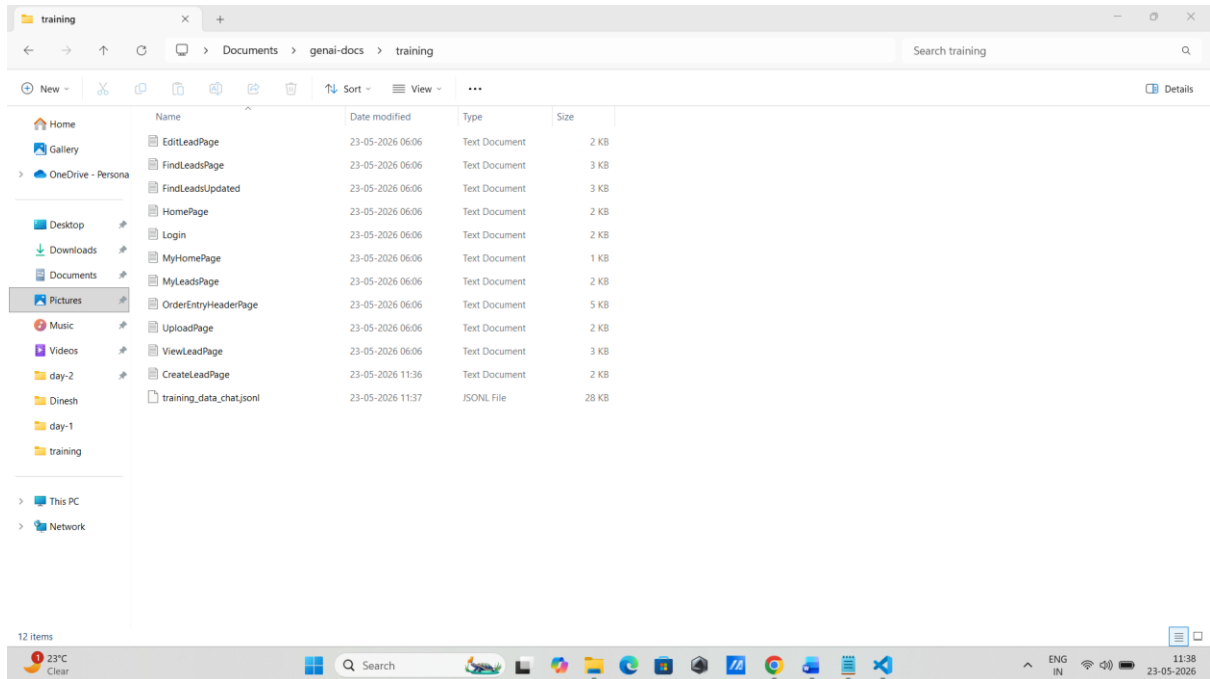
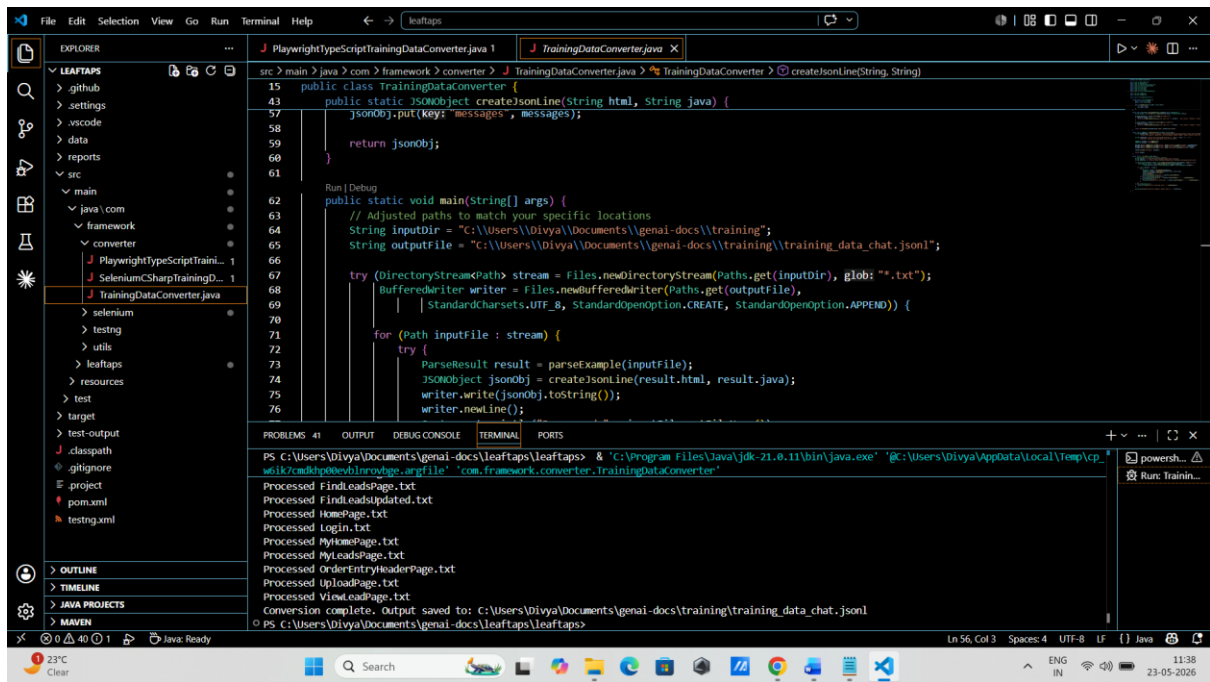
    }

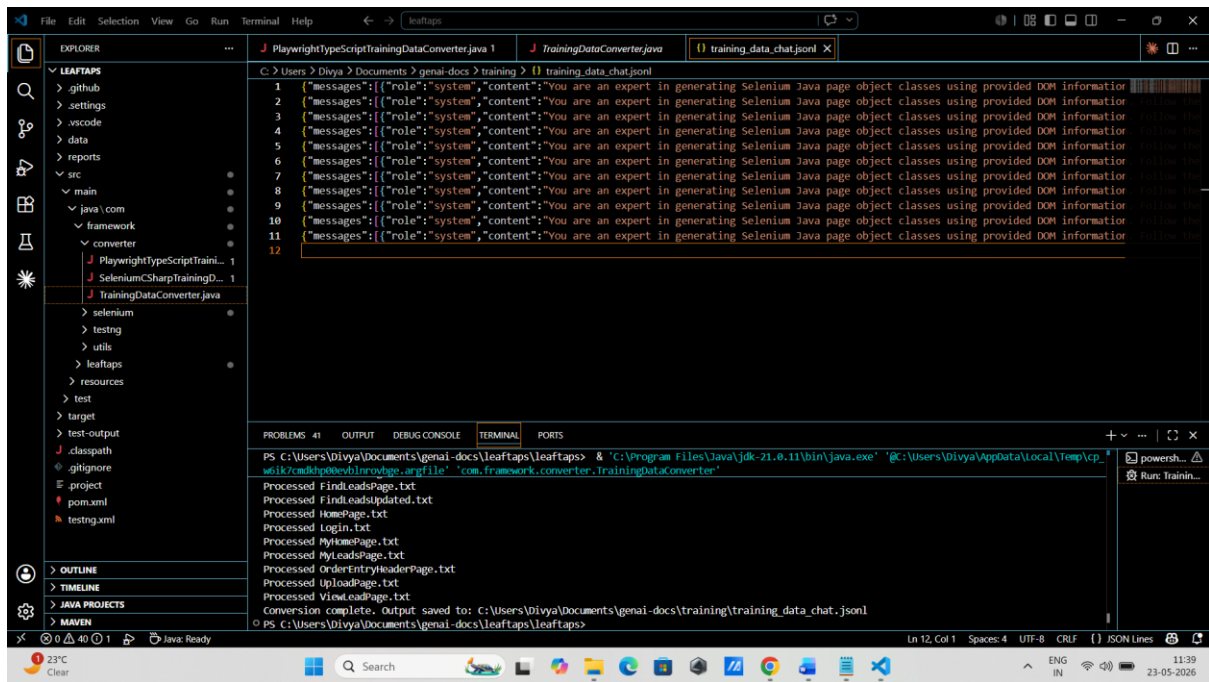
```

```

}

```





```
{
  "messages": [
    {
      "role": "system",
      "content": "You are an expert in generating Selenium Java page object classes using provided DOM information. Follow the framework's guidelines: use SeleniumBase wrapper methods, proper locators, and insert TODO comments if a mapping is missing. Make sure every page method includes comments in your generated code as the training examples may not have comments."
    },
    {
      "role": "user",
      "content": "Given the following DOM structure:\n` `` html\n<input type=\"text\" class=\"inputBox\" name=\"companyName\" size=\"30\" maxLength=\"100\" id=\"createLeadForm_companyName\">\n<input type=\"text\" class=\"inputBox\" name=\"firstName\" size=\"30\" maxLength=\"100\" id=\"createLeadForm_firstName\">\n<input type=\"text\" class=\"inputBox\" name=\"lastName\" size=\"30\" maxLength=\"100\" id=\"createLeadForm_lastName\">\n<input type=\"submit\" class=\"smallSubmit\" name=\"submitButton\" value=\"Create Lead\" id=\"ext-gen652\">\n` ``\nGenerate a Selenium Java page object class for this DOM."
    }
  ]
}
```

```

},

{
    "role": "assistant",

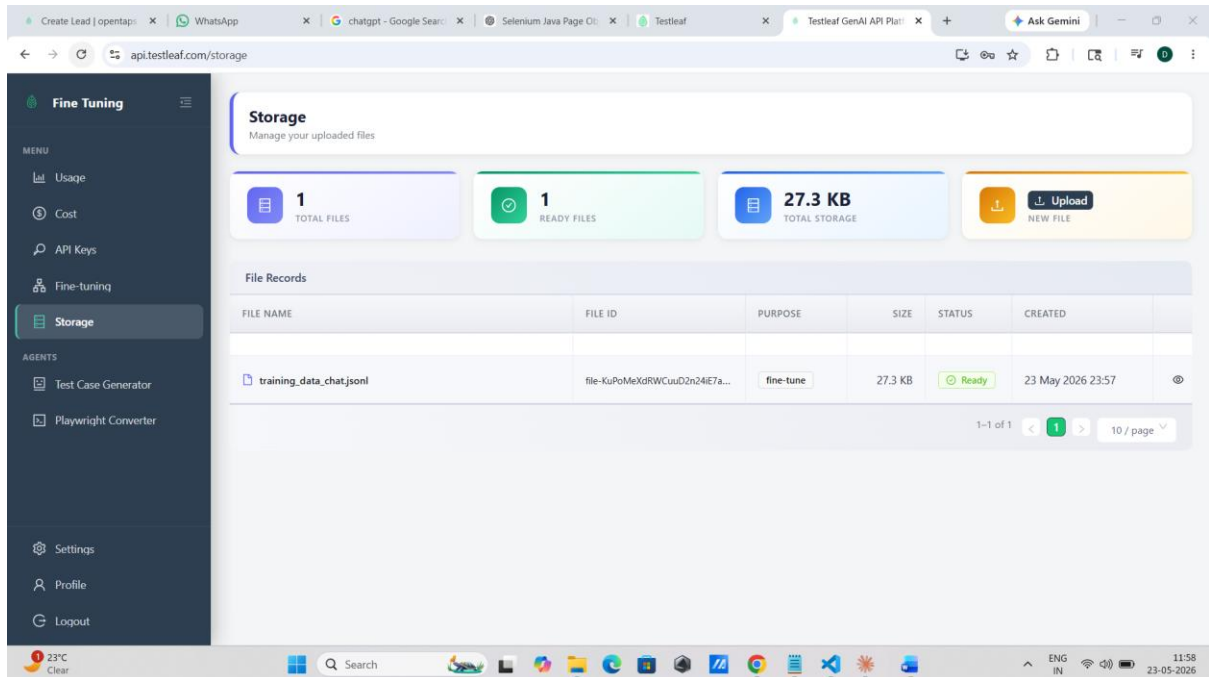
    "content": "package com.leaf taps.pages;\n\nimport
com.framework.selenium.api.design.Locators;\nimport
com.framework.testng.api.base.ProjectSpecificMethods;\n\n/**\n * Page Object Class
for Create Lead Page.\n * \n * This class contains all the web elements and reusable
methods\n * related to the Create Lead functionality.\n * \n * @author Auto-generated\n
*/\npublic class CreateLeadPage extends ProjectSpecificMethods {\n\n    /**\n     *
Enters the company name in the Company Name field.\n     * \n     * @param
companyName - Name of the company\n     * @return CreateLeadPage - Current page
instance\n     */\n    public CreateLeadPage enterCompanyName(String companyName)
{\n\n        // Type company name into Company Name textbox\n        type(\n
locateElement(Locators.ID, \"createLeadForm_companyName\"),\n
companyName\n    );\n\n        // Reporting step for success\n        reportStep(\n
\"Company name entered successfully as: \" + companyName,\n            \"pass\"\n
);\n\n        return this;\n    }\n\n    /**\n     * Enters the first name in the First Name field.\n
     * \n     * @param firstName - First name of the lead\n     * @return CreateLeadPage -
Current page instance\n     */\n    public CreateLeadPage enterFirstName(String
firstName) {\n\n        // Type first name into First Name textbox\n        type(\n
locateElement(Locators.ID, \"createLeadForm_firstName\"),\n            firstName\n
);\n\n        // Reporting step for success\n        reportStep(\n            \"First name entered
successfully as: \" + firstName,\n                \"pass\"\n        );\n\n        return this;\n    }\n\n    /**\n     * Enters the last name in the Last Name field.\n
     * \n     * @param lastName - Last name of the lead\n     * @return CreateLeadPage - Current page instance\n     */\n    public CreateLeadPage enterLastName(String
lastName) {\n\n        // Type last name into Last Name textbox\n        type(\n
locateElement(Locators.ID,
\"createLeadForm_lastName\"),\n            lastName\n        );\n\n        // Reporting step for
success\n        reportStep(\n            \"Last name entered successfully as: \" +
lastName,\n                \"pass\"\n        );\n\n        return this;\n    }\n\n    /**\n     * Clicks the Create Lead button.\n
     * \n     * @return ViewLeadPage - Navigates to View Lead Page\n     */\n    public ViewLeadPage clickCreateLead() {\n\n        // Click on Create Lead button\n
click(\n            locateElement(Locators.CLASS_NAME, \"smallSubmit\")\n        );\n\n        // Reporting step for success\n        reportStep(\n            \"Create Lead button clicked
successfully\",\n                \"pass\"\n        );\n\n        return new ViewLeadPage();\n    }\n
}

}

}

```

2) Go to api.testleaf.com ---Menu --Storage --upload the dataset (copy the file id - file-R3jftEJrhj2fSMbATZr8k9)



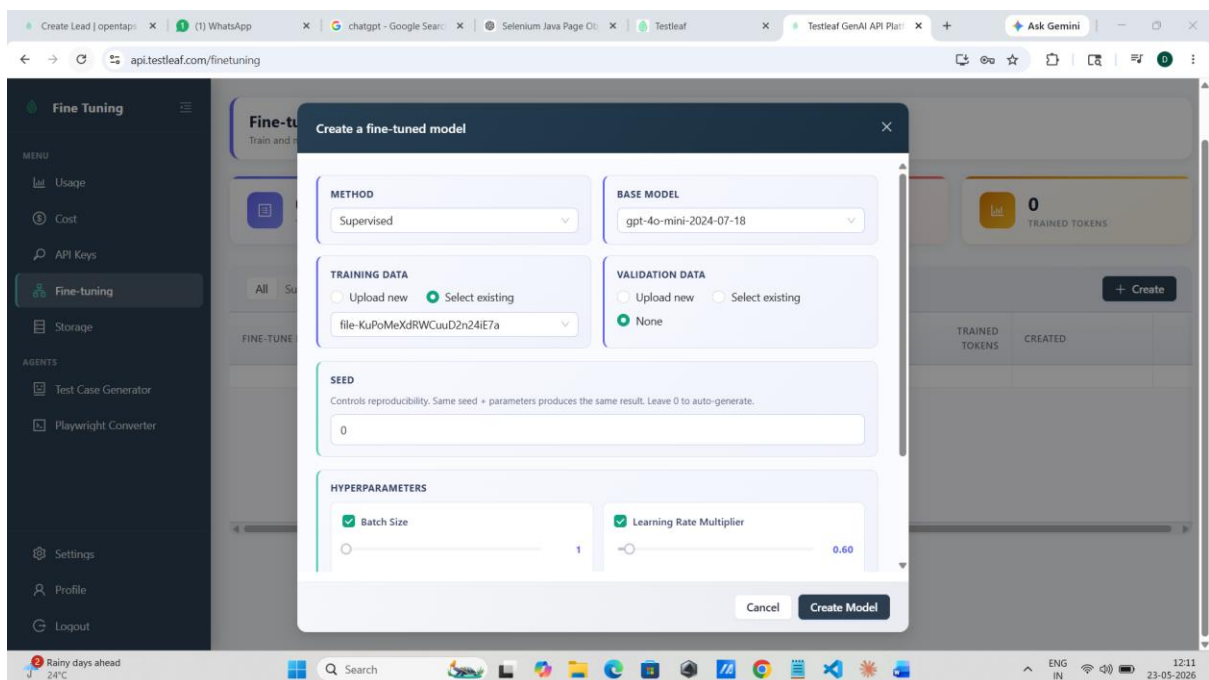
File ID- file-KuPoMeXdRWCuuD2n24iE7a

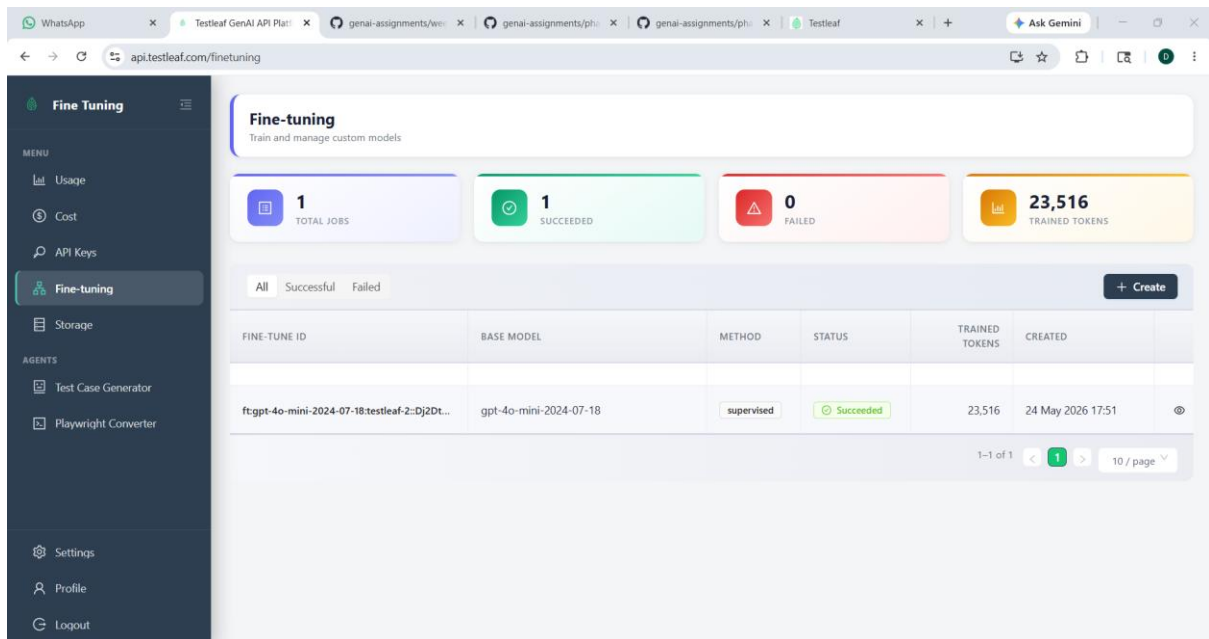
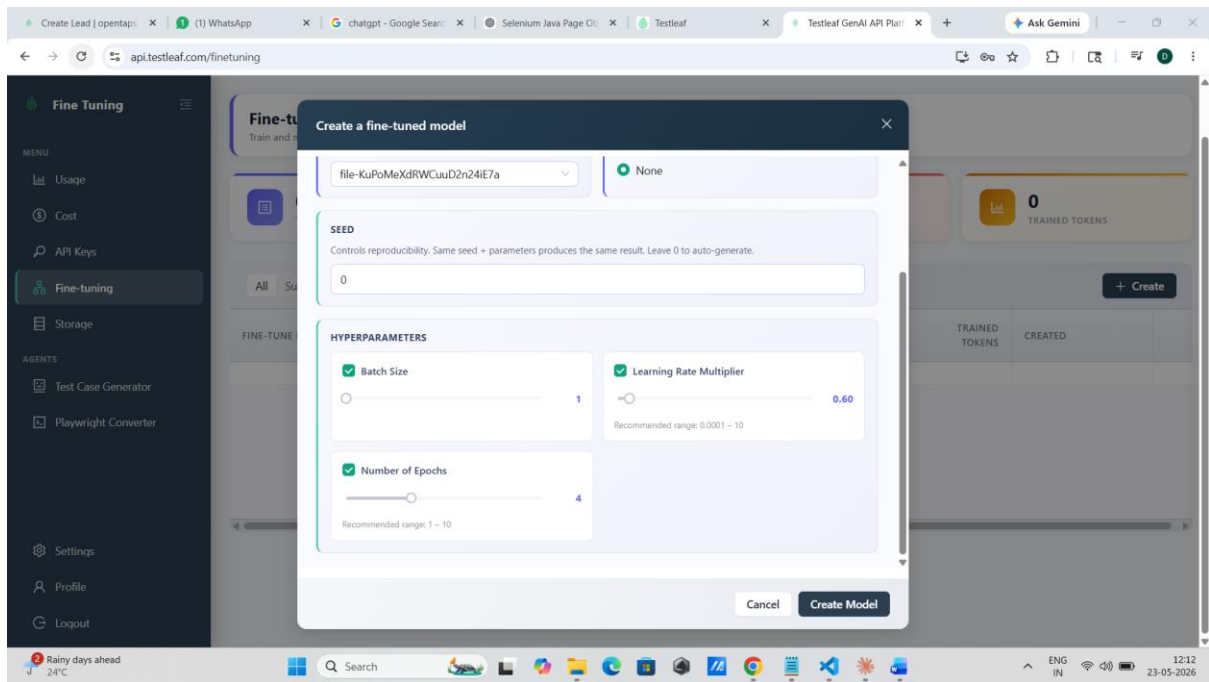
3) Go to Finetuning menu

select existing data(File ID will be populated)

Learning multiplier-0.5 or 0.6

Epochs -4 or 5

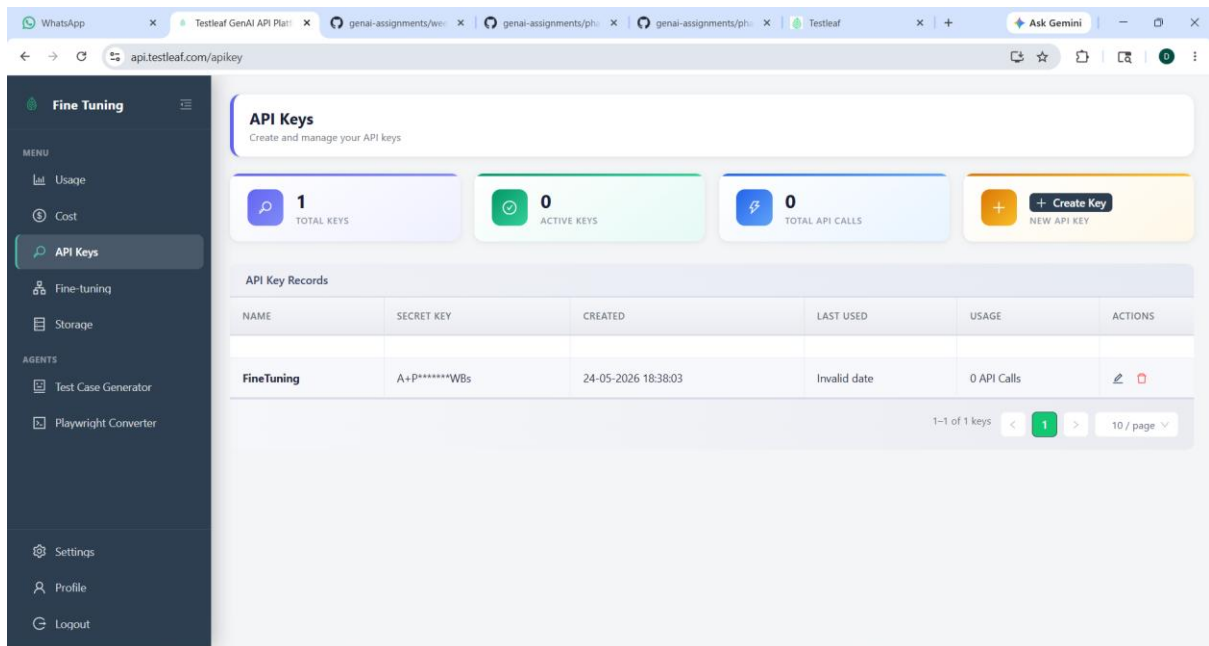




Copy the Fine tuned model

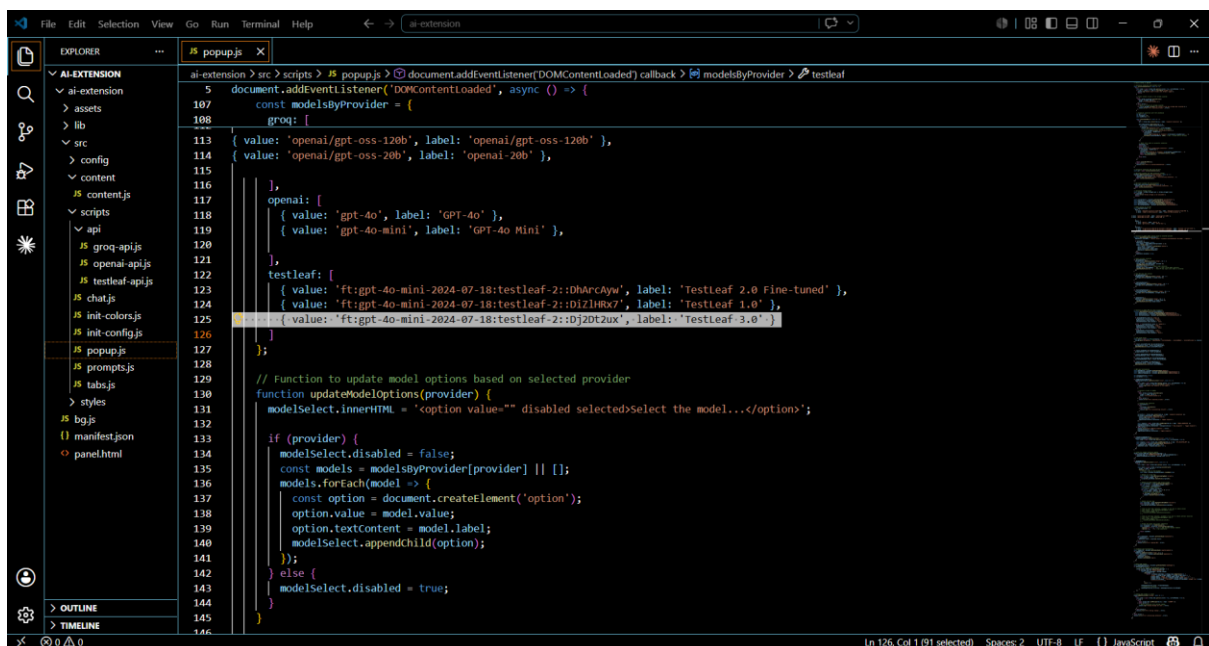
ft:gpt-4o-mini-2024-07-18:testleaf-2::Dj2Dt2ux

4)Generate the Key (api.testleaf.com)



1) Current AI Extension used in browser ,open this file in vscode

2) In popup.js implement code for using testleaf api



Add the testleaf api key in the key in teh extension

API Key-YfbHczNvLEKb

Code Generator

Settings

Supported Language

TypeScript

Browser Engine

Playwright

Select LLM Provider

Testleaf

Select Model

TestLeaf 3.0

Testleaf API Key

.....

2) WhatsApp

Testleaf GenAI AP...

genai-assignment...

genai-assignment...

genai-assignment...

Testleaf

Amazon Sign-In

Ask Gemini

amazon.com/ap/signin?openid.return_to=https%3A%2F%2Fwww.amazon.com%2F%3Fref_%3Dnav_ya_signin&openid.identity=http%3A%2F%2Fspecs.openid.net%2Fauth%2F2.0...

amazon

Sign in or create account

Enter mobile number or email

Continue

By continuing, you agree to Amazon's [Conditions of Use](#) and [Privacy Notice](#).

[Need help?](#)

Buying for work?

Create a free business account

Conditions of Use

Privacy Notice

Help

© 1996–2026, Amazon.com, Inc. or its affiliates

AI Browser Extension for Code Generati...

Code Generator

Settings

☐ Feature File

☒ Page Class

Reset Chat

```
* @param countryCode - The country
*/
async selectCountryCode(countryCode:
  await this.countryDropdown.select0
)

/**
 * Clears the email or mobile input
 */
async clearEmailOrMobileInput(): Pro
  await this.clearButton.click();
}

/**
 * Checks if the error alert is visi
 * @returns True if the error alert
 */
async isErrorAlertVisible(): Promise
  return await this.errorAlert.isVis
}
}
```

```

import { Page, Locator } from '@playwright/test';

/**
 * Page Object for Claim Collection Page
 */
export class ClaimCollectionPage {
  page: Page;
  private emailInput: Locator;
  private passwordInput: Locator;
  private continueButton: Locator;
  private countryDropdown: Locator;
  private clearButton: Locator;
  private errorAlert: Locator;

  constructor(page: Page) {
    this.page = page;
    this.emailInput = page.locator('#ap_email_login');
    this.passwordInput = page.locator('#auth-credential-autofill-hint');
    this.continueButton = page.locator('#continue');
    this.countryDropdown = page.locator('#claim-input-dropdown-select-element');
    this.clearButton = page.locator('#claim-input-clear-button');
    this.errorAlert = page.locator('#error-alert');
  }

  /**
   * Sets the email or mobile number in the input field.
   * @param emailOrMobile - The email or mobile number to enter.

```

```
*/  
  
async enterEmailOrMobile(emailOrMobile: string): Promise<void> {  
    await this.emailInput.fill(emailOrMobile);  
}
```

```
/**  
  
 * Sets the password in the input field.  
 * @param password - The password to enter.  
 */
```

```
async enterPassword(password: string): Promise<void> {  
    await this.passwordInput.fill(password);  
}
```

```
/**  
  
 * Clicks the continue button to submit the form.  
 */
```

```
async clickContinueButton(): Promise<void> {  
    await this.continueButton.click();  
}
```

```
/**  
  
 * Selects a country code from the dropdown.  
 * @param countryCode - The country code to select.  
 */  
  
async selectCountryCode(countryCode: string): Promise<void> {  
    await this.countryDropdown.selectOption(countryCode);  
}
```

```

/**
 * Clears the email or mobile input field.
 */
async clearEmailOrMobileInput(): Promise<void> {
    await this.clearButton.click();
}

/**
 * Checks if the error alert is visible.
 * @returns True if the error alert is visible, otherwise false.
 */
async isErrorAlertVisible(): Promise<boolean> {
    return await this.errorAlert.isVisible();
}
}

```

1) Current AI Extension used in browser ,open this file in vscode

2) In popup.js implement code for using testleaf api

```

const modelsByProvider = {
    groq: [
        { value: 'deepseek-r1-distill-llama-70b', label: 'deepseek-r1-distill-llama-70b' },
        { value: 'llama-3.3-70b-versatile', label: 'llama-3.3-70b-versatile' },

        { value: 'openai/gpt-oss-120b', label: 'openai/gpt-oss-120b' },
        { value: 'openai/gpt-oss-20b', label: 'openai-20b' },
    ],
}

```

```

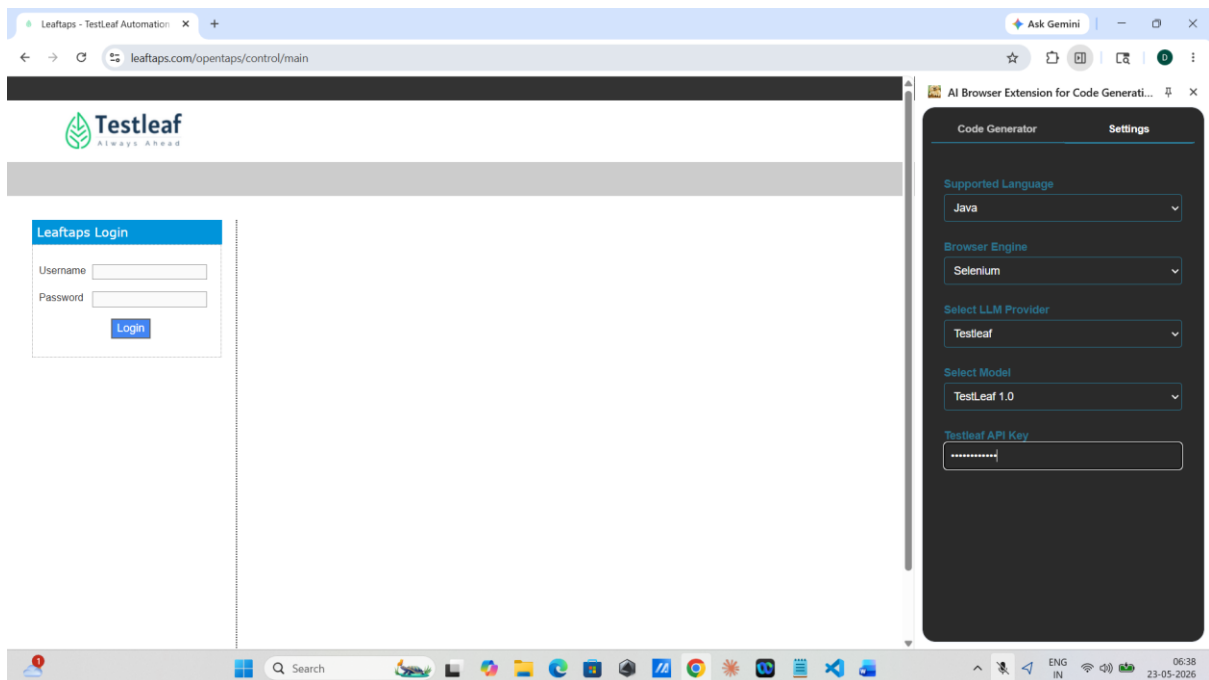
],
openai: [
  { value: 'gpt-4o', label: 'GPT-4o' },
  { value: 'gpt-4o-mini', label: 'GPT-4o Mini' },

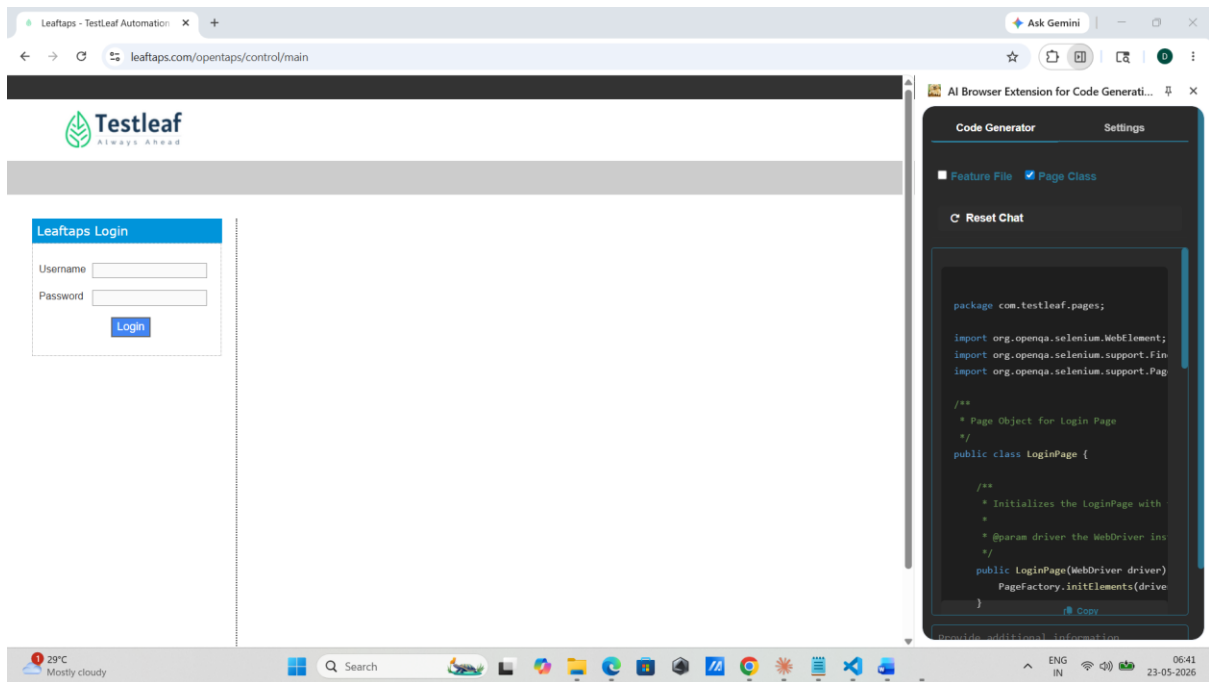
],
testleaf: [
  { value: 'ft:gpt-4o-mini-2024-07-18:testleaf-2::DhArcAyw', label: 'TestLeaf 2.0 Fine-tuned' },
  { value: 'ft:gpt-4o-mini-2024-07-18:testleaf-2::DiZlHRx7', label: 'TestLeaf 1.0' }
]
};

```

Add the testleaf api key in the key in teh extension

API Key-YfbHczNvLEKb





```
package com.testleaf.pages;
```

```
import org.openqa.selenium.WebElement;
```

```
import org.openqa.selenium.support.FindBy;
```

```
import org.openqa.selenium.support.PageFactory;
```

```
/**
```

```
 * Page Object for Login Page
```

```
 */
```

```
public class LoginPage {
```

```
/**
```

```
 * Initializes the LoginPage with the given driver.
```

```
 *
```

```
 * @param driver the WebDriver instance
```

```
 */
```

```
public LoginPage(WebDriver driver) {
```

```
PageFactory.initElements(driver, this);  
}
```

```
@FindBy(id = "username")  
private WebElement usernameField;
```

```
@FindBy(id = "password")  
private WebElement passwordField;
```

```
@FindBy(className = "decorativeSubmit")  
private WebElement loginButton;
```

```
/**  
 * Enters the username in the login form.  
 *  
 * @param username the username to enter  
 * @return the current LoginPage instance  
 */  
public LoginPage enterUsername(String username) {  
    usernameField.clear();  
    usernameField.sendKeys(username);  
    return this;  
}
```

```
/**  
 * Enters the password in the login form.  
 *  
 * @param password the password to enter
```

```
* @return the current LoginPage instance
*/

public LoginPage enterPassword(String password) {

    passwordField.clear();

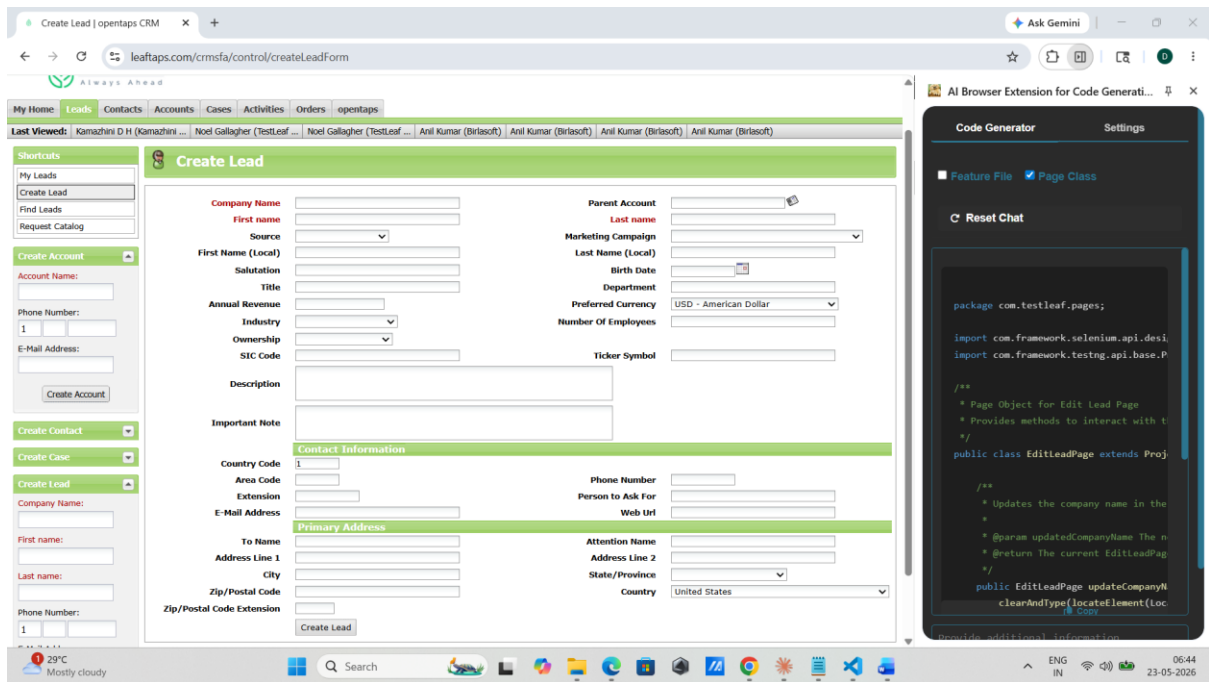
    passwordField.sendKeys(password);

    return this;
}


/**
 * Clicks the login button to submit the form.
 *
 * @return the HomePage after login
 */
public HomePage clickLogin() {

    loginButton.click();

    return new HomePage(driver);
}
}
```



```
package com.testleaf.pages;
```

```
import com.framework.selenium.api.design.Locators;
```

```
import com.framework.testng.api.base.ProjectSpecificMethods;
```

```
/**
```

```
 * Page Object for Edit Lead Page
```

```
 * Provides methods to interact with the Edit Lead form fields and actions.
```

```
 */
```

```
public class EditLeadPage extends ProjectSpecificMethods {
```

```
 /**
```

```
 * Updates the company name in the Edit Lead form.
```

```
 *
```

```
 * @param updatedCompanyName The new company name to enter
```

```
 * @return The current EditLeadPage instance
```

```
 */
```

```

public EditLeadPage updateCompanyName(String updatedCompanyName) {
    clearAndType(locateElement(Locators.ID,"updateLeadForm_companyName"),
updatedCompanyName);

    reportStep(updatedCompanyName + " is entered as Company Name", "pass");
    return this;
}

/**
 * Clicks the Update button to submit the form.
 *
 * @return A new ViewLeadPage instance
 */
public ViewLeadPage clickUpdateButton() {
    click(locateElement(Locators.XPATH,"//input[@value='Update']"));
    return new ViewLeadPage();
}
}

```